

SCRIPT-CONTROLLED ACL - RESTRICTED RECORD

ACCESS BASED ON FIELD VALUE

A Naan Mudhalvan Report submitted in partial fulfillment of the requirements for the award
Degree in

BACHELOR OF SCIENCE DEGREE IN COMPUTER SCIENCE.

MANONMANIAM SUNDARANAR UNIVERSITY,

TIRUNELVELI.



SUBMITTED BY:

NAME

REGISTER NO

M.ASIFA

24083281802112110

J.ATHILA RAIHAN

24083281802112112

M.IHSANA

24083281802112118

V.S MONISHA

24083281802112127

N.PREETHI SRI

24083281802112130

DEPARTMENT OF COMPUTER SCIENCE

WOMEN'S CHRISTIAN COLLEGE, NAGERCOIL

NOVEMBER 2026

CERTIFICATE

This is to certify that the Naan Mudhalvan work entitled “**SCRIPT-CONTROLLED ACL - RESTRICTED RECORD ACCESS BASED ON FIELD VALUE** “ is the Bonafide work
done by

NAME

REGISTER NO

M.ASIFA
J.ATHILA RAIHAN
M.IHSANA
V.S MONISHA
N.PREETHI SRI

24083281802112110
24083281802112112
24083281802112118
24083281802112127
24083281802112130

In partial fulfilment for the award of degree of Bachelor of Science in
Manonmaniam Sundaranar University during the academic year 2026-2027

Head of the Department

Place: Nagercoil

Date:

Submitted for the viva-voice Examination held at Women’s Christian College on
_____ Internal Examiner External Examiner

CONTENTS

1.INTRODUCTION

2.IDEATION PHASE

3. REQUIREMENT ANALYSIS

4. PROJECT DESIGN

5. PROJECT PLANNING & SCHEDULING

6. FUNCTIONAL AND PERFORMANCE TESTING

7. RESULTS

8. ADVANTAGES & DISADVANTAGES

9. CONCLUSION

ABSTRACT

SCRIPT-CONTROLLED ACL – RESTRICTED RECORD ACCESS BASED ON FIELD VALUE is a security-focused project designed to control access to records based on specific field values. The system uses Access Control Lists (ACLs) alongwith scripting logic to determine whether a user is permitted to view or access particular records. Instead of providing the same level of access to all users, the system evaluates predefined field values and dynamically applies access restrictions according to the user's role or authorization.

The main objective of this project is to improve data security and prevent unauthorized access to sensitive records. When a user attempts to access a record, the script checks the relevant field value and applies the appropriate access rule. This approach provides more flexible and granular control than traditional role-based access alone. The system can be used in applications where different users require access to different records based on departments, categories, status, ownership, or other field-level conditions.

The proposed system demonstrates how script-controlled ACLs can provide dynamic, reliable, and manageable record-level security. It helps organizations protect sensitive information while allowing authorized users to access the records required for their work.

1. INTRODUCTION

1.1 Project Overview

Script-Controlled ACL – Restricted Record Access Based on Field Value is a ServiceNow-based project developed in the Skill Wallet workspace. The project demonstrates how Access Control Lists (ACLs) and server-side scripting can be combined to control access to individual records according to the value of a selected field. Instead of giving the same access level to every record, the access decision can be evaluated dynamically when a user attempts to read or interact with a record.

The main idea is to make record access field-dependent. For example, records can be made accessible only when a particular field satisfies a defined condition. The ACL script evaluates the current record and the user context, and the request is allowed or denied according to the configured rule. This provides a practical demonstration of fine-grained security in the ServiceNow platform.

1.2 Purpose

The purpose of this project is to understand and implement dynamic record-level security using ServiceNow ACLs and scripting. It aims to demonstrate how a security rule can be applied based on the value stored in a record field rather than relying only on a general table or role-level permission.

- To understand the role of ACLs in ServiceNow security.
- To restrict access to records according to a field value.
- To use server-side scripting as part of an ACL condition.

- To test both permitted and restricted access scenarios.
- To provide a simple and reusable model for field-based record security.

2. IDEATION PHASE

2.1 Problem Statement

In a ServiceNow application, users may have permission to access a table while not necessarily being authorized to view every record in that table. A basic table-level permission may therefore be too broad for situations where records contain different levels of accessibility. The problem addressed by this project is how to restrict individual records dynamically according to the value of a field.

The proposed approach uses a scripted ACL. When a user attempts to access a record, the ACL evaluates the relevant field value and determines whether the operation should be permitted. This makes the security logic more specific than a simple unrestricted table-level rule.

2.2 Empathy Map Canvas

The empathy map for the project considers the needs of a ServiceNow administrator or application developer who must protect records without creating a large number of separate access rules.

- Says: The user needs controlled access to sensitive or restricted records.
- Thinks: Access should be automatic, consistent, and easy to maintain.
- Does: Configures tables, fields, ACLs and scripts and tests access with different records.
- Feels: Concerned about unauthorized access and incorrect security configuration.
- Needs: A clear rule that links record access to a field value.
-

- Pain Point: A broad permission may expose records that should remain restricted.

2.3 Brainstorming

During brainstorming, several possible approaches were considered, including role-based access, condition-based ACLs, scripted ACLs and combinations of roles with record conditions. The selected approach focuses on a script-controlled ACL because scripting provides flexibility when the access decision depends on the current record and its field values.

- Role-based access for general permissions.
- Condition-based ACL for simple field conditions.
- Scripted ACL for dynamic record-level decisions.
- Testing with records containing different field values.
- Combining user permissions with record conditions where required.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

The user journey begins when an authorized user signs in to the ServiceNow environment and navigates to the relevant application or workspace. The user then attempts to open a record. At this point, the configured ACL is evaluated. If the record satisfies the defined field-value rule and the user has the required base permission, access is granted. Otherwise, access to the restricted record is denied.

- Login / authentication → user enters the ServiceNow environment.
- Navigation → user opens the required application or workspace.
- Record request → user attempts to access a record.

- Security evaluation → ACL and script evaluate the request.
- Decision → access is allowed or restricted.
- User feedback → permitted data is displayed; restricted data is not exposed.

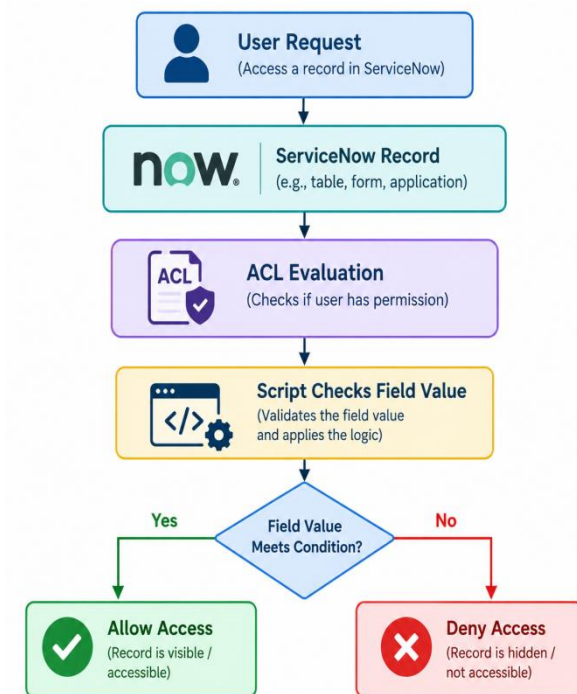
3.2 Solution Requirement

The solution requires a ServiceNow environment in which the relevant table and field are available and an ACL can be configured. The ACL must be associated with the appropriate operation, such as read access, and the script must evaluate the target record's field value. The solution should also include test records that represent both allowed and restricted conditions.

- ServiceNow instance / Skill Wallet workspace.
- Required table and target field.
- Appropriate user or role permissions.
- Record-level ACL.
- Server-side ACL script.
- Test records with different field values.
- A method to verify allowed and denied access.

3.3 Data Flow Diagram

The basic data flow of the project can be represented as follows:



When the user requests a record, ServiceNow invokes the relevant access control mechanism. The ACL checks whether the user is eligible for the requested operation. The scripted condition then evaluates the selected field value of the current record. The final result determines whether the record can be accessed.

3.4 Technology Stack

- Platform: ServiceNow
- Workspace: Skill Wallet workspace
- Security mechanism: Access Control List (ACL)
- Scripting: ServiceNow server-side JavaScript
- Data: ServiceNow table records and field values
- Testing: ServiceNow user/role and record-access testing

4. PROJECT DESIGN

4.1 Problem Solution

The project fits the identified problem because it moves record authorization from a broad permission model towards a more granular field-based decision. The ACL remains part of the platform's standard security model, while the script provides the logic required to examine the current record. This approach can be adapted to different fields and business conditions without changing the basic architecture.

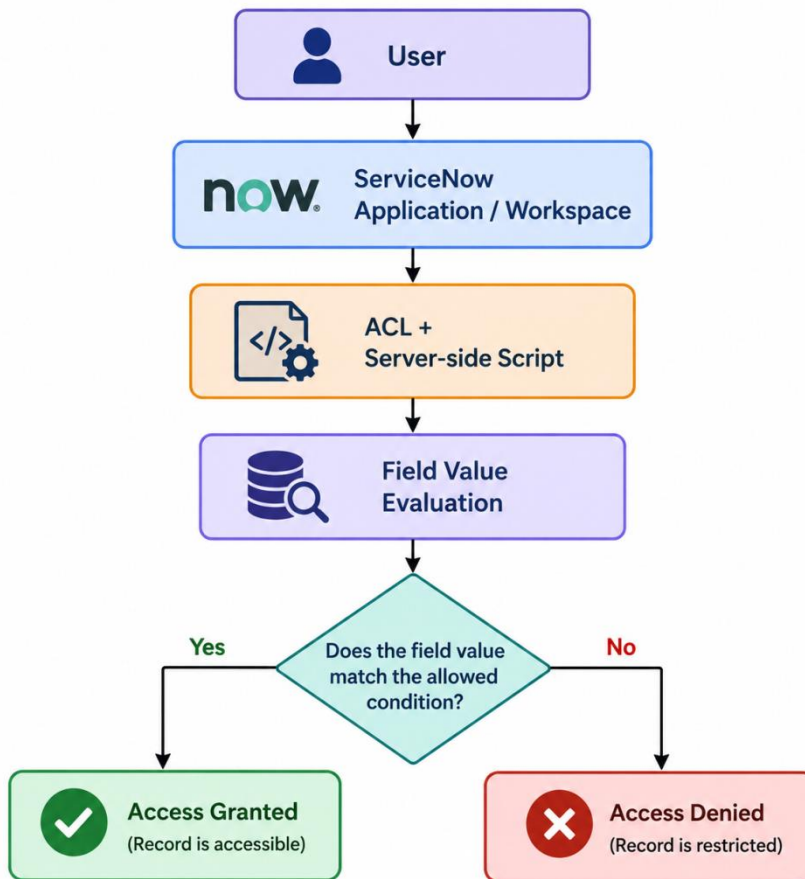
4.2 Proposed Solution

The proposed solution is a scripted record-level ACL. The ACL is configured for the required operation and uses a server-side script to evaluate the value of a selected field in the current record. When the condition matches the permitted value, the ACL allows the operation. When the condition does not match, the operation is restricted.

The exact field and permitted value depend on the implementation in the Skill Wallet workspace. This report therefore describes the reusable design rather than assuming a field name or value that is not provided in the project details.

4.3 Solution Architecture

The architecture contains four main layers. The user layer initiates the record request. The ServiceNow application layer receives the request and identifies the target record. The security layer evaluates the ACL and executes the script. Finally, the data layer supplies the record and its field value for evaluation.



5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

The project was planned as a sequence of requirement identification, security-rule design, configuration, scripting, testing and documentation. The development process can be summarized in the following stages:

- Stage 1 – Identify the access-control problem and select a field-based rule.
- Stage 2 – Identify the ServiceNow table, target field and required operation.
- Stage 3 – Configure the ACL with the appropriate access requirement.
- Stage 4 – Implement the server-side script for field-value evaluation.
- Stage 5 – Create or identify test records representing permitted and restricted values.

- Stage 6 – Test the ACL with appropriate users/roles and record conditions.
- Stage 7 – Document the configuration, results and future improvements.

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

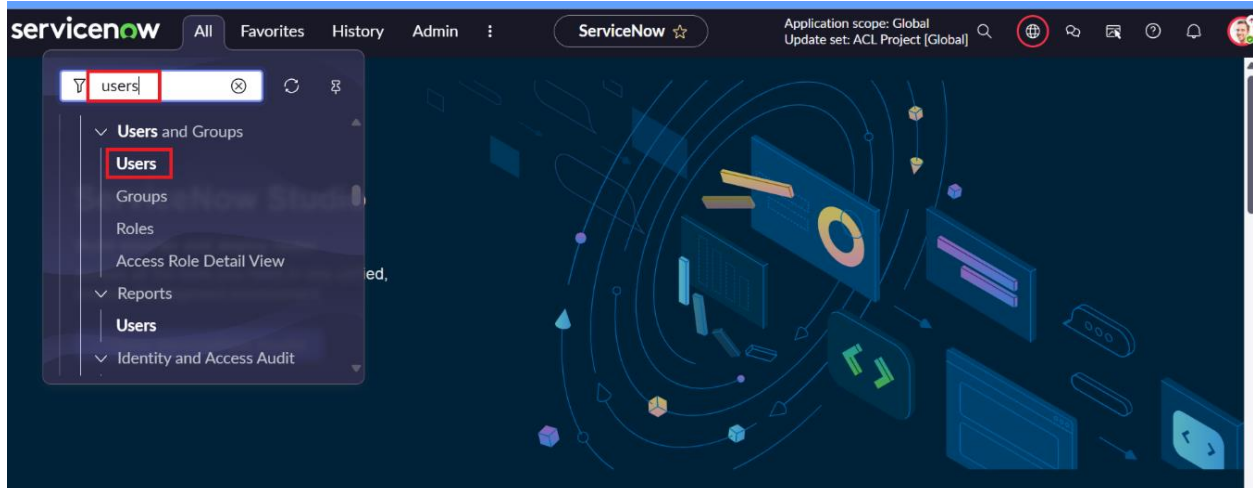
Performance testing for this project focuses on verifying that the scripted ACL evaluates access without introducing unnecessary processing. The script should use a simple and direct condition based on the current record's field value. Testing should be performed with representative records and repeated access attempts to confirm consistent behavior.

Functional testing should verify at least two cases: a record whose field value satisfies the configured condition and a record whose field value does not satisfy it. The first case should produce the permitted result, while the second should produce the restricted result. Additional tests can verify behavior for empty, unexpected or changed field values where relevant to the implementation.

7. RESULTS

Milestone-1 Creation of Users and Roles

Users Creation



Roles Creation

A screenshot of the ServiceNow 'Role' creation form. The form is titled 'Role BB1'. It contains a 'Name' field with 'BB1' entered, an 'Application' dropdown set to 'Global', and an 'Elevated privilege' checkbox. A 'Description' text area is also present. At the bottom, there are 'Update' and 'Delete' buttons. The 'Update' button is highlighted with a red box. The form is part of the 'SN Utils | Versions (1)' section.

Milestone-2 Table creation

A screenshot of the ServiceNow 'institution Details' table. The table has a search bar at the top with 'Student Roll Number' selected. The table contains the following data:

Student Roll Number	Faculty name	Student Name	Branch	Email	Phone Number
INS0001006	ITIL User	Praneeth	ECE	praneeth@gmail.com	8989090906
INS0001008	Malla Sravan kumar	Abraham Lincoln	CSE	sravan@gmail.com	8978908989
INS0001005	ITIL User	Srinu Nandipalli	ECE	srinu@gmail.com	9089909090
INS0001007	Abel Tuter	Guna Sai	EEE	salguna@gmail.com	9090898765
INS0001001	Abraham Lincoln	Yaswanth Nikku	CSE	yaswanth@gmail.com	8989898989
INS0001004	Abel Tuter	Prashanth Reddy	EEE	prashanth@gmail.com	9090909090

Milestone-3 Creation of Access Control list-READ

Access Control
u_institution_details

* Type: record

* Operation: read

Decision Type: Allow If

Admin overrides: ☒

Protection policy: -- None --

* Name: institution Details [u_institution_details]

Description: </>

Application: Global

Active: ☒

Advanced: ☒

Update Delete

Milestone-4 Creation of Access Control list-CREATE

Access Control
u_institution_details

* Type: record

* Operation: create

Decision Type: Allow If

Admin overrides: ☒

Protection policy: -- None --

* Name: institution Details [u_institution_details]

Description: </>

Applies To: No. of records matching the condition: 6

Add Filter Condition Add OR Clause

-- choose field -- -- oper -- -- value --

Application: Global

Active: ☒

Advanced: ☐

Update Delete

Milestone-5. Creation of Access Control list-WRITE

Access Control
u_institution_details

* Type: record

* Operation: write

Decision Type: Allow If

Admin overrides: ☒

Protection policy: -- None --

* Name: institution Details [u_institution_details]

Description: </>

Applies To: No. of records matching the condition: 6

Add Filter Condition Add OR Clause

-- choose field -- -- oper -- -- value --

Application: Global

Active: ☒

Advanced: ☐

Update Delete

Milestone-6 Creation of Access Control list-DELETE

Access Control New record

Warning: A role, security attribute, data condition, script or ACL control via reference fields is required to properly secure access with this ACL.

* Type: record

* Operation: delete

Decision Type: Allow If

Application: Global

Active: ☒

Advanced: ☐

Admin overrides: ☒

Protection policy: -- None --

* Name: Institution Details [u_institution_details]

Description:

Applies To: No. of records matching the condition: 6

Add Filter Condition Add OR Clause

-- choose field -- -- oper -- -- value --

8. ADVANTAGES & DISADVANTAGES

Advantages

- Provides fine-grained record-level access control.
- Uses standard ServiceNow ACL security mechanisms.
- Can evaluate the current record dynamically.
- Reduces the need for separate rules for every individual record.
- Can be adapted to different fields and business conditions.
- Helps prevent unauthorized access to restricted records.

Disadvantages

- Incorrect ACL configuration can unintentionally restrict or expose records.
- Complex scripts can become difficult to maintain.
- Testing is necessary whenever the field-value rule changes.
- Access decisions may become harder to troubleshoot if many ACLs interact.

- The solution depends on correct ServiceNow roles, permissions and ACL configuration.

9. CONCLUSION

The Script-Controlled ACL project demonstrates how ServiceNow can be used to implement dynamic and granular record security. By combining an Access Control List with server-side scripting, the project restricts or permits access according to a field value in the current record. This provides a practical solution for situations where users should not receive identical access to every record in a table.

The project also provides practical understanding of ServiceNow security configuration, ACL evaluation, server-side scripting and testing. The same design can be extended to more complex business rules while keeping the security decision close to the data-access layer.

10. FUTURE SCOPE

- Extend the rule to support multiple field values and multiple access conditions.
- Combine field-based rules with user roles or groups.
- Introduce more detailed audit and monitoring of access decisions.
- Create reusable security logic for multiple tables or applications where appropriate.
- Add automated test cases for different users and record conditions.
- Integrate more complex business rules while keeping ACL scripts maintainable.

11. APPENDIX

Source Code

1. General ACL Definition Fields (For each ACL record)

For every ACL you created in your milestones (Read, Create, Write, Delete), document these

core fields:

- ☐ Type: Record
- ☐ Operation: Select the matching operation (Read, Create, Write, or Delete).
- ☐ Name / Table: Your target custom or base table (e.g., u_your_table_name or incident) followed by <table>.none for record-level security.
- ☐ Active: true (Checked)
- ☐ Admin Overrides: Checked (allows admins to bypass the rule).

2. Advanced Scripts by Operation

- ☐ For READ / WRITE / DELETE ACLs (Field Value Condition):

JavaScript

```
// Checks a field value on the existing record (current)
```

```
if (current.u_field_name == '&#39;target_value&#39;) {
```

```
    answer = false; // Deny access
```

```
} else {  
  
answer = true; // Allow access  
  
}
```

❑ For CREATE ACLs (Checking parent/field values before creation):

JavaScript

```
// On creation, current object exists, but fields might be evaluated
```

```
before insert
```

```
if (current.u_field_name == 'restricted_value') {
```

```
answer = false;
```

```
} else {
```

```
answer = true;
```

```
}
```

Dataset Link

The project uses sample records created within the ServiceNow Skill Wallet workspace for testing the Script-Controlled ACL. The access decision is based on the value of the specified field in the ServiceNow record.

GitHub & Project Demo Link

GitHub Repository: [https://github.com/ihsana7957-ux/script-controller-](https://github.com/ihsana7957-ux/script-controller-ACL)

ACL Project Demo / Skill Wallet Link:

<https://drive.google.com/file/d/1UWtBovEOHEAdLGKKmyBt7IJFLEJnTSKj/view?usp=sharing>